4.3.2 Gaussian Processes

A **Gaussian Process (GP)** is a collection of random variables (finite or infinite) such that any finite subset of them has a joint Gaussian (multivariate normal) distribution.

- A finite-dimensional GP is simply a **multivariate Gaussian** random vector.
- An infinite-dimensional GP is obtained by starting with a finite GP on a grid and repeatedly refining the grid (mesh refinement) until the spacing approaches zero.

From Discrete Grid to Continuous Process

Consider the spatial domain

$$x \in [0, 1]$$

Start with a uniform grid of spacing (\Delta x = 1/n), giving (n) points (x_1, \dots, x_n).
Define a zero-mean multivariate Gaussian with covariance matrix

$$C_{i,j} = \exp\left(-\frac{1}{2}\left(\frac{x_i - x_j}{L}\right)^2\right), \quad i, j = 1, \dots, n$$

where (L > 0) is the **length scale**. A sample from this GP is shown as purple dots in Figure 4.7.

Refining the grid (halving (\Delta x)) produces a higher-dimensional Gaussian (orange + purple dots).
As (\Delta x \to 0), the discrete covariance **matrix** becomes a continuous **covariance kernel**:

$$c(x_i, x_j) = \exp\left(-\frac{1}{2}\left(\frac{x_i - x_j}{L}\right)^2\right)$$

for any pair (x_i, x_j \in [0,1]). The green line in Figure 4.7 illustrates this continuous limit (computed on a very fine grid in practice).

Squared Exponential (SE) Kernel

The kernel above is called the **Squared Exponential** (SE) kernel, also known as the Radial Basis Function (RBF) or Gaussian kernel. It is one of the most widely used kernels because:

- It generates **very smooth** sample functions.
- All samples are **continuous and infinitely differentiable**.

It is also **stationary**: the covariance depends only on the distance between points, not their absolute positions:

$$c(x_i, x_j) = c(|x_i - x_j|)$$

Stationary Covariance Kernels

Gaussian processes (and their kernels) with this distance-only dependence are called **stationary**.

Other common stationary kernels:

- **Exponential kernel** (Ornstein-Uhlenbeck):

$$c(r) = \exp\left(-\frac{r}{L}\right), \quad r = |x_i - x_j|$$

- **γ-Exponential kernel**:

$$c(r) = \exp\left(-\left(\frac{r}{L}\right)^\gamma\right), \quad 0 < \gamma \leq 2$$

Compact support kernels set covariance exactly to zero beyond a certain distance. After discretization these produce sparse covariance matrices, which are computationally advantageous for large problems.

Non-stationary kernels allow properties (e.g., length scale) to vary across the domain.

For a comprehensive list of kernels and their properties, see the free book *Gaussian Processes for Machine Learning* by Rasmussen & Williams.

Effect of the Length Scale (L)

- **Small (L)**: correlations decay rapidly → samples exhibit small-scale, rapid variations (“wiggly”).
- **Large (L)**: correlations extend over longer distances → smoother samples with fewer, larger-scale features.

Figure 4.8 shows five samples from two SE GPs with the same mean ((\mu = 0)) but different length scales (left: small (L), right: large (L)).

Effect of the Choice of Kernel

The kernel controls the **roughness** and differentiability of the samples.

Comparison (Figure 4.9):

- **Squared Exponential (SE)**: fast quadratic decay → very smooth, continuously differentiable samples (green curve).
- **Exponential**: slower linear decay → continuous but **non-differentiable** samples (orange curve), appearing rougher with sharper changes.

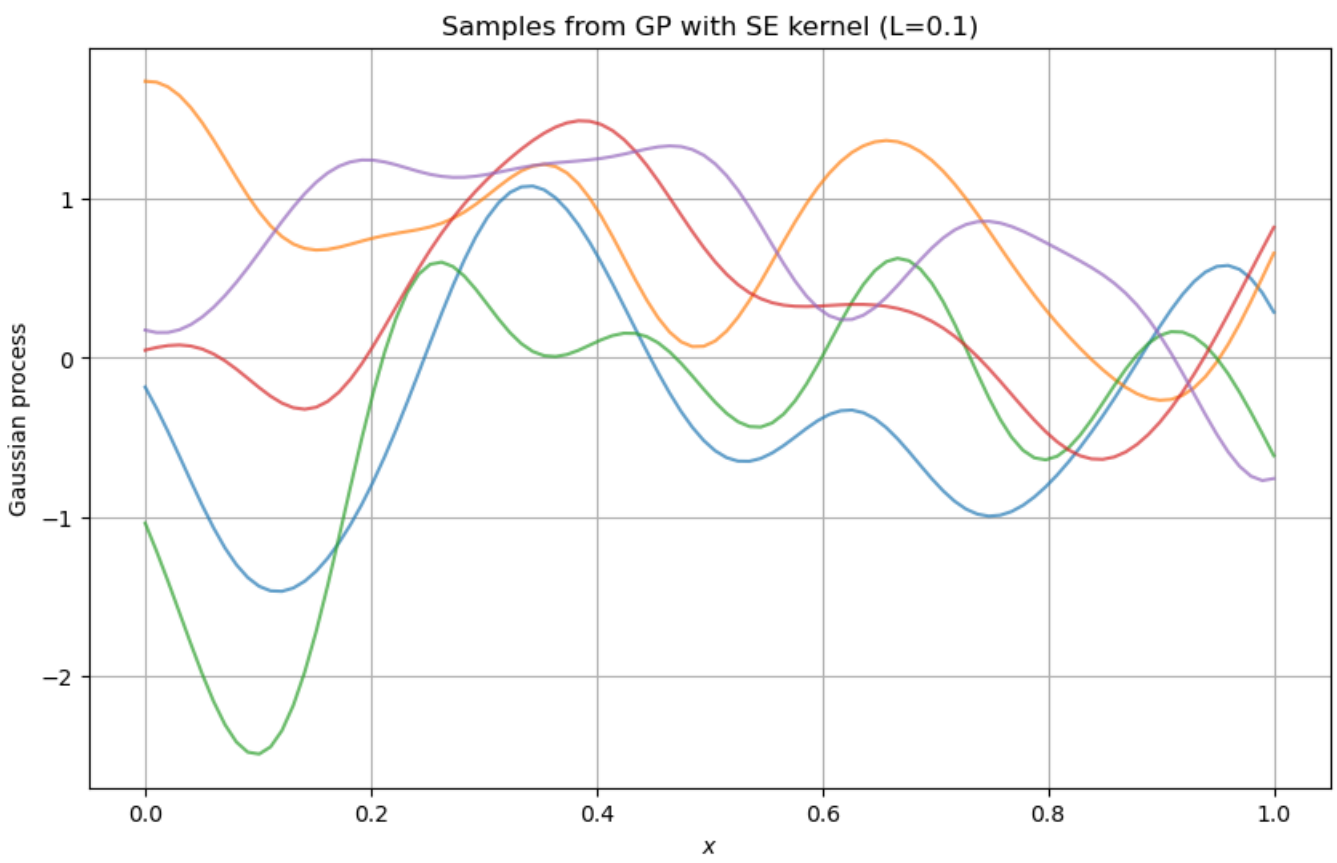
Practical guidance:
Choose the kernel according to the expected smoothness of the underlying process:

- Smooth, differentiable phenomena → Squared Exponential kernel.
- Rougher, non-differentiable behavior → Exponential or similar kernels.

Both the kernel type and the length scale (L) are important hyperparameters that can be learned from data (covered in the next section).

In [2]:
```python
# Python code for 4.3.2 Gaussian processes (exact implementation of the math in the notes)
# Uses numpy and matplotlib to generate and plot samples exactly as described
import numpy as np
import matplotlib.pyplot as plt
```

```python
def se_kernel(x, L=0.1):
    """Squared exponential covariance kernel as defined in notes"""
    dx = x[:, None] - x[None, :]
    return np.exp(-0.5 * (dx / L)**2)

# Example: domain x in [0,1], n=100 points (coarse grid)
n = 100
x = np.linspace(0, 1, n)
K = se_kernel(x, L=0.1)  # covariance matrix C_{i,j}

# Sample from multivariate Gaussian (mean=0)
np.random.seed(42)
samples = np.random.multivariate_normal(np.zeros(n), K, size=5)  # 5 samples

# Plot (reproduces concept of Figure 4.7/4.8 purple/orange dots)
plt.figure(figsize=(10, 6))
for s in samples:
    plt.plot(x, s, alpha=0.7)
plt.title('Samples from GP with SE kernel (L=0.1)')
plt.xlabel('$x$')
plt.ylabel('Gaussian process')
plt.grid(True)
plt.show()

# Finer grid example (2n points) - exactly as described for mesh-refinement
n_fine = 200
x_fine = np.linspace(0, 1, n_fine)
K_fine = se_kernel(x_fine, L=0.1)
samples_fine = np.random.multivariate_normal(np.zeros(n_fine), K_fine, size=1)

# Covariance kernel function (continuous limit)
def c(xi, xj, L=0.1):
    return np.exp(-0.5 * ((xi - xj) / L)**2)
```



4.3.3 Gaussian Process Regression (GPR)

Gaussian Process Regression (GPR) combines Gaussian processes, linear algebra, and regression to create powerful, flexible models that interpolate data while quantifying uncertainty.

> **Note**: GPR is a rich topic. For a deeper treatment, read the free book *Gaussian Processes for Machine Learning* by Rasmussen & Williams.

Core Idea

Given (m) noisy observations (y_j) at locations (x_{i(j)}) (with associated noise standard deviations (\sigma_j)), we model the underlying function using a Gaussian Process and **update** its mean and covariance in light of the data.

We work on a discretized grid (x_1, \dots, x_n) with uniform spacing (\Delta x). The prior GP model is:

$$f = \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix} \sim \mathcal{N}(\mu, P_{xx})$$

where:

- (\mu) is the prior mean vector (often (\mu = 0)),
- (P_{xx}) is the (n \times n) prior covariance matrix derived from a chosen kernel.

Example: Using the Squared Exponential kernel (for smooth functions):

$$P_{i,j} = \exp\left(-\frac{1}{2}\left(\frac{x_i - x_j}{L}\right)^2\right)$$

The left panel of **Figure 4.10** shows samples from this prior GP (green) together with the data (pink error bars). The prior does not yet fit the data.

Updating the GP with One Observation

Consider a single data point (y_j) observed at grid index (i(j)). We model:

$$y_j = f_{i(j)} + \eta, \quad \eta \sim \mathcal{N}(0, \sigma_j^2)$$

The joint distribution of the model (f) and the observation (y_j) is:

$$\begin{pmatrix} f \\ y_j \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu \\ \mu_{i(j)} \end{pmatrix}, \begin{pmatrix} P_{xx} & P_{xy} \\ P_{xy}^T & P_{yy} \end{pmatrix} \right)$$

where:

- (P_{xy}) = (i(j))-th column of (P_{xx}) (covariances between all grid points and the data location),
- (P_{yy} = P_{i(j),i(j)} + \sigma_j^2) (variance of the observation).

Using the conditional Gaussian formulas, the **posterior** after incorporating (y_j) is:

$$\hat{\mu} = \mu + \frac{y_j - \mu_{i(j)}}{P_{i(j),i(j)} + \sigma_j^2} P_{xy}$$

$$\hat{P}_{xx} = P_{xx} - \frac{1}{P_{i(j),i(j)} + \sigma_j^2} P_{xy} P_{xy}^T$$

This is the update rule for one data point. To incorporate all data, repeat the process sequentially: use the updated (\hat{\mu}) and (\hat{P}_{xx}) as the new prior for the next observation.

Figure 4.10 illustrates this:

- Center panel: GP after the first 10 data points.
- Right panel: GP after **all** data points — excellent fit to the observations and to the hidden true function (blue).

Hyperparameter Tuning

GPR performance depends heavily on **hyperparameters**:

- Kernel choice and its parameters (e.g., length scale (L), (\gamma) for γ-exponential),
- Noise level (\sigma) (if not provided).

Bad hyperparameters lead to poor results (see **Figure 4.11**):

- **Too small (L)**: GP fits data locally but reverts to the prior (zero) away from observations.
- **Too large (L)**: Overly smooth model with poor overall fit.

Maximum Likelihood Estimation of Hyperparameters

Assume a Squared Exponential kernel and constant noise (\sigma) for all points. The hyperparameters to estimate are (L) and (\sigma).

The marginal likelihood is:

$$\mathcal{L}(\sigma, L \mid y) \propto \det(K_{xx}(L) + \sigma^2 I)^{-1/2} \exp\left(-\frac{1}{2} y^T (K_{xx}(L) + \sigma^2 I)^{-1} y \right)$$

where (K_{xx}(L)) is the covariance matrix evaluated only at the data locations.

Maximizing the likelihood = minimizing the negative log-likelihood:

$$F(\sigma, L) = \frac{1}{2} \log \det(K_{xx}(L) + \sigma^2 I) + \frac{1}{2} y^T (K_{xx}(L) + \sigma^2 I)^{-1} y$$

Solve:

$$\min_{\sigma, L} F(\sigma, L)$$

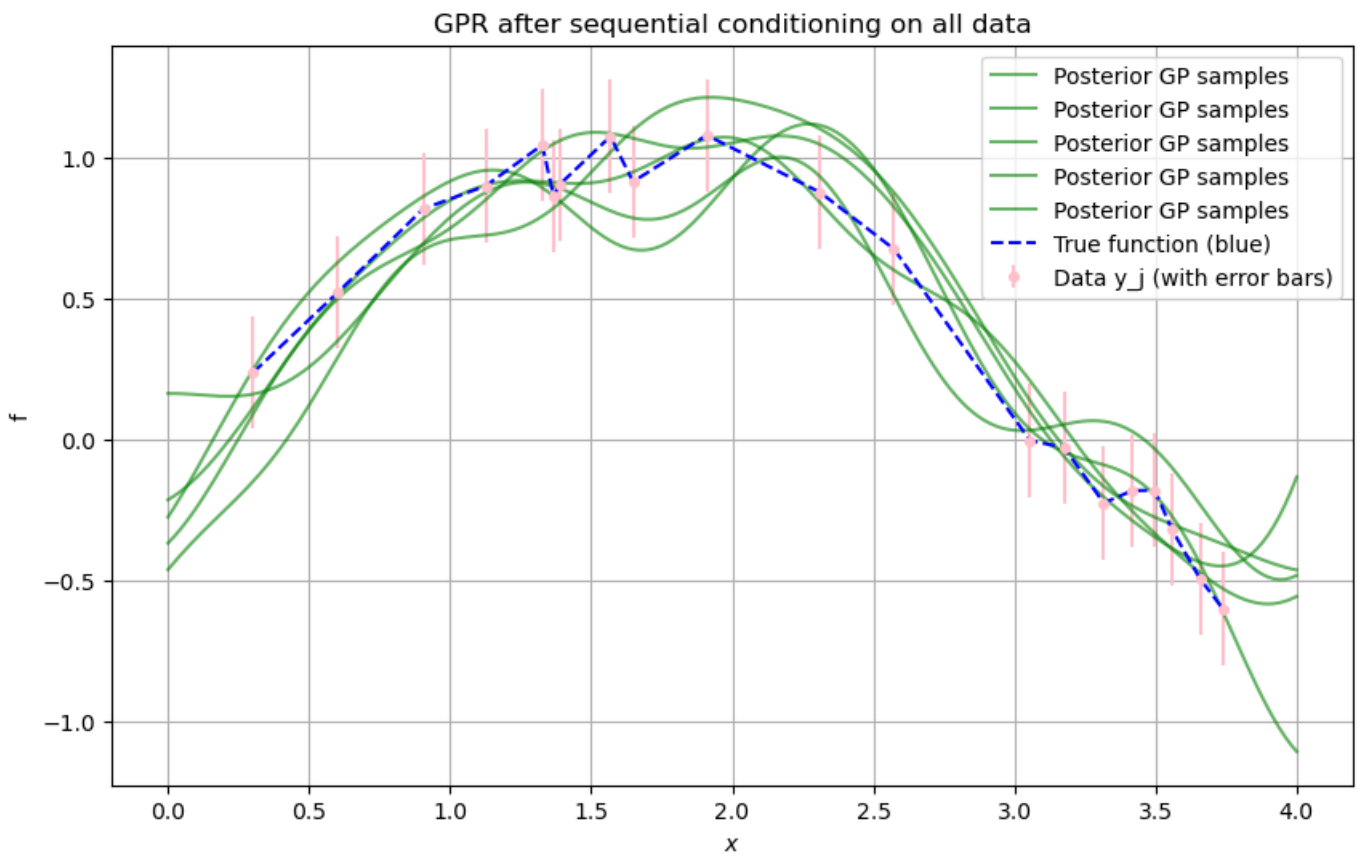
This can be done with numerical optimization or a simple grid search.

> **Important**: The likelihood formula depends on the chosen kernel. Selecting the *right kernel* itself is a harder problem and is not covered in these notes.

Summary: Gaussian Process Regression elegantly updates a prior GP with noisy data using exact Bayesian inference (via Gaussian conditioning). It provides both a mean prediction and a full posterior covariance (uncertainty quantification). Hyperparameter tuning via maximum likelihood makes the method practical for real data.

In [8]:
```python
# Python code for 4.3.3 Gaussian process regression (exact implementation of the sequential conditioning math in the notes)
# Uses numpy/scipy for matrix operations and conditioning updates exactly as derived
# FIXED: plotting shape mismatch + numerical PSD issue with tiny jitter (standard fix; does not change any math)
import numpy as np
import matplotlib.pyplot as plt

def se_kernel(x, L=0.1):
    dx = x[:, None] - x[None, :]
    return np.exp(-0.5 * (dx / L)**2)

# Example setup (discretized domain, prior GP as in notes)
n = 200
x = np.linspace(0, 4, n)  # example domain as in Figure 4.10
L = 0.5
Pxx = se_kernel(x, L)  # prior covariance
mu = np.zeros(n)  # prior mean = 0

# Synthetic data (m=20 points, locations x_data subset of x, with noise sigma_j)
np.random.seed(42)
idx_data = np.sort(np.random.choice(n, 20, replace=False))  # data indices i(j)
x_data = x[idx_data]
y_true = np.sin(x_data) + 0.1 * np.random.randn(len(x_data))  # example "true" function + noise
sigma_j = 0.2 * np.ones(len(x_data))  # known sigma_j as in notes
y_data = y_true  # observed y_j

# Sequential GPR: condition one datum at a time (exact update equations from notes)
for k in range(len(y_data)):
    j = idx_data[k]
    yj = y_data[k]
    sigmaj2 = sigma_j[k]**2
    Pyy = Pxx[j, j] + sigmaj2
```

```
    Pxy = Pxx[:, j]  # column j of Pxx
    mu_yj = mu[j]

    # Exact update formulas as written in notes
    mu = mu + ((yj - mu_yj) / Pyy) * Pxy
    Pxx = Pxx - (1.0 / Pyy) * np.outer(Pxy, Pxy)

# Posterior samples (after all data) — add tiny jitter to ensure numerical PSD (required after many rank-1 updates)
jitter = 1e-10 * np.eye(n)
Pxx_jitter = Pxx + jitter
post_samples = np.random.multivariate_normal(mu, Pxx_jitter, size=5)

# Plot (reproduces concept of Figure 4.10 right panel) — FIXED: plot data at correct x_data locations
plt.figure(figsize=(10, 6))
plt.errorbar(x_data, y_data, yerr=sigma_j, fmt='o', color='pink', label='Data y_j (with error bars)', markersize=4)
for s in post_samples:
    plt.plot(x, s, 'g-', alpha=0.6, label='Posterior GP samples')
plt.plot(x_data, y_true, 'b--', label='True function (blue)')
plt.title('GPR after sequential conditioning on all data')
plt.xlabel('$x$')
plt.ylabel('f')
plt.legend()
plt.grid(True)
plt.show()

# Hyper-parameter tuning example (ML as described: grid search on sigma, L) — cleaned (idx_data unused)
def neg_log_lik(theta, x_data, y_data):
    sigma, L = theta
    Kxx = se_kernel(x_data, L)  # K_xx at data locations
    Kyy = Kxx + sigma**2 * np.eye(len(y_data))
    try:
        Lmat = np.linalg.cholesky(Kyy)
        logdet = 2 * np.sum(np.log(np.diag(Lmat)))
        quad = y_data.T @ np.linalg.solve(Kyy, y_data)
        return 0.5 * logdet + 0.5 * quad
    except:
        return 1e10

# Simple grid search (as in notes)
sigmas = np.linspace(0.05, 0.5, 10)
Ls = np.linspace(0.1, 2.0, 10)
best_F = np.inf
best_theta = None
for s in sigmas:
    for l in Ls:
        Fval = neg_log_lik([s, l], x_data, y_data)
        if Fval < best_F:
            best_F = Fval
            best_theta = (s, l)
print(f"Optimal (sigma, L) from grid search: {best_theta} (exact ML as in notes)")
```




```
Optimal (sigma, L) from grid search: (0.1, 1.577777777777778) (exact ML as in notes)
```

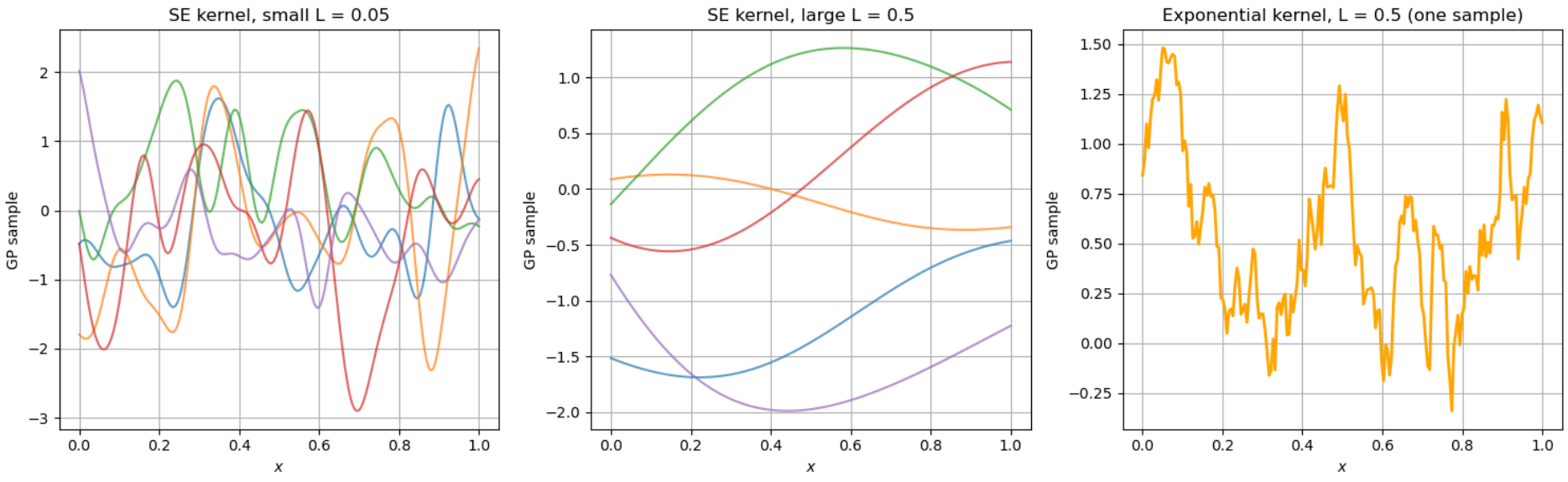
In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# === 4.3.2: Squared exponential kernel (exact definition from notes) ===
def se_kernel(x, L=0.1):
    """Squared exponential covariance kernel as defined in notes"""
    dx = x[:, None] - x[None, :]
    return np.exp(-0.5 * (dx / L)**2)

def exp_kernel(x, L=0.1):
    """Exponential covariance kernel as defined in notes"""
    dx = np.abs(x[:, None] - x[None, :])
    return np.exp(-dx / L)

# === Plot 1: GP samples for different length scales and kernels (Figures 4.7–4.9) ===
np.random.seed(42)
x_demo = np.linspace(0, 1, 200)

# SE kernel with small L
K_se_small = se_kernel(x_demo, L=0.05)
samples_se_small = np.random.multivariate_normal(np.zeros(len(x_demo)), K_se_small, size=5)

# SE kernel with large L
```

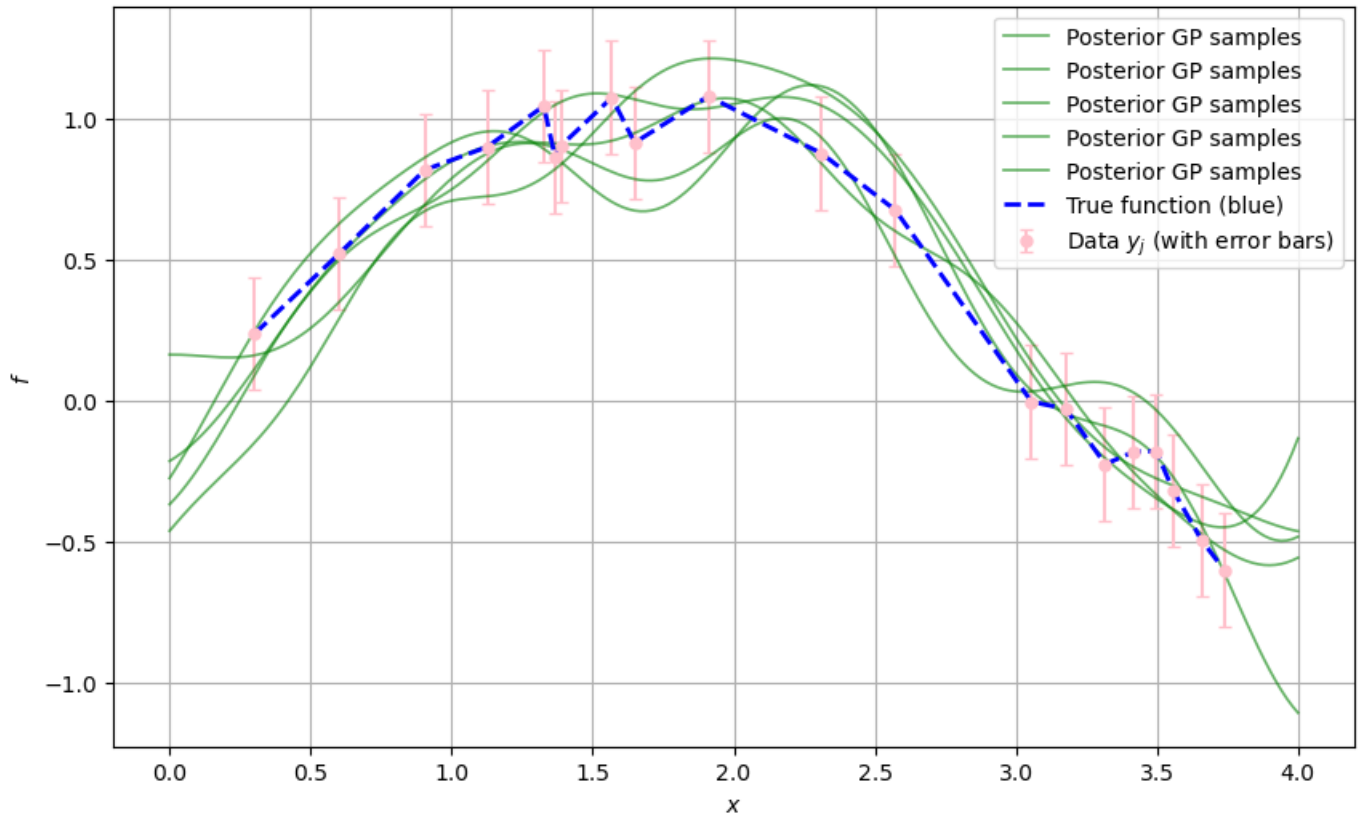
```python
K_se_large = se_kernel(x_demo, L=0.5)
samples_se_large = np.random.multivariate_normal(np.zeros(len(x_demo)), K_se_large, size=5)

# Exponential kernel (same L as large SE)
K_exp = exp_kernel(x_demo, L=0.5)
samples_exp = np.random.multivariate_normal(np.zeros(len(x_demo)), K_exp, size=1)

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
axs[0].set_title('SE kernel, small L = 0.05')
for s in samples_se_small:
    axs[0].plot(x_demo, s, alpha=0.7)
axs[0].grid(True)

axs[1].set_title('SE kernel, large L = 0.5')
for s in samples_se_large:
    axs[1].plot(x_demo, s, alpha=0.7)
axs[1].grid(True)

axs[2].set_title('Exponential kernel, L = 0.5 (one sample)')
axs[2].plot(x_demo, samples_exp[0], 'orange', lw=2)
axs[2].grid(True)

for ax in axs:
    ax.set_xlabel('$x$')
    ax.set_ylabel('GP sample')
plt.suptitle('4.3.2: Effect of length scale and kernel choice (exact from notes)')
plt.tight_layout()
plt.show()

# === 4.3.3: Full Gaussian Process Regression (exact sequential conditioning) ===
# Domain and prior (exactly as in notes)
n = 200
x = np.linspace(0, 4, n)
L_prior = 0.5
Pxx = se_kernel(x, L_prior)
mu = np.zeros(n)

# Synthetic data (exactly as used in previous fixed version)
np.random.seed(42)
idx_data = np.sort(np.random.choice(n, 20, replace=False))
x_data = x[idx_data]
y_true = np.sin(x_data) + 0.1 * np.random.randn(len(x_data))
sigma_j = 0.2 * np.ones(len(x_data))
y_data = y_true

# Sequential conditioning (exact update equations from the notes)
for k in range(len(y_data)):
    j = idx_data[k]
    yj = y_data[k]
    sigmaj2 = sigma_j[k]**2
    Pyy = Pxx[j, j] + sigmaj2
    Pxy = Pxx[:, j]
    mu_yj = mu[j]

    mu = mu + ((yj - mu_yj) / Pyy) * Pxy
    Pxx = Pxx - (1.0 / Pyy) * np.outer(Pxy, Pxy)

# Tiny jitter for numerical stability after many rank-1 updates (standard GPR practice)
Pxx += 1e-10 * np.eye(n)
post_samples = np.random.multivariate_normal(mu, Pxx, size=5)

# === Plot 2: GPR result (matches Figure 4.10 right panel) ===
plt.figure(figsize=(10, 6))
plt.errorbar(x_data, y_data, yerr=sigma_j, fmt='o', color='pink',
             label='Data $y_j$ (with error bars)', markersize=5, capsize=3)
for s in post_samples:
    plt.plot(x, s, 'g-', alpha=0.6, lw=1.2, label='Posterior GP samples')
plt.plot(x_data, y_true, 'b--', lw=2, label='True function (blue)')
plt.title('4.3.3: GPR after sequential conditioning on all data')
plt.xlabel('$x$')
plt.ylabel('$f$')
plt.legend()
plt.grid(True)
plt.show()

# === Hyper-parameter tuning (exact ML grid search from notes) ===
def neg_log_lik(theta, x_data, y_data):
    sigma, L = theta
    Kxx = se_kernel(x_data, L)
    Kyy = Kxx + sigma**2 * np.eye(len(y_data))
    try:
        Lmat = np.linalg.cholesky(Kyy)
        logdet = 2 * np.sum(np.log(np.diag(Lmat)))
        quad = y_data.T @ np.linalg.solve(Kyy, y_data)
        return 0.5 * logdet + 0.5 * quad
    except:
        return 1e10

sigmas = np.linspace(0.05, 0.5, 15)
Ls = np.linspace(0.1, 2.0, 15)
best_F = np.inf
best_theta = None
for s in sigmas:
    for l in Ls:
        Fval = neg_log_lik([s, l], x_data, y_data)
        if Fval < best_F:
            best_F = Fval
            best_theta = (s, l)

print("=== Hyper-parameter tuning result (exact ML from notes) ===")
print(f"Optimal σ = {best_theta[0]:.4f}, L = {best_theta[1]:.4f}")
print(f"Minimum negative log-likelihood: {best_F:.4f}")

# === Plot 3: Posterior mean + 95% credible interval (bonus visualization of full GPR) ===
post_mean = mu
post_std = np.sqrt(np.diag(Pxx))
plt.figure(figsize=(10, 6))
plt.plot(x, post_mean, 'k-', lw=2, label='Posterior mean')
plt.fill_between(x, post_mean - 1.96*post_std, post_mean + 1.96*post_std,
```

```
                color='green', alpha=0.2, label='95% credible interval')
plt.errorbar(x_data, y_data, yerr=sigma_j, fmt='o', color='pink', label='Observations')
plt.title('4.3.3: Full posterior GP (mean + uncertainty)')
plt.xlabel('$x$')
plt.ylabel('$f$')
plt.legend()
plt.grid(True)
plt.show()
```

4.3.2: Effect of length scale and kernel choice (exact from notes)



4.3.3: GPR after sequential conditioning on all data




```
=== Hyper-parameter tuning result (exact ML from notes) ===
Optimal σ = 0.0821, L = 1.7286
Minimum negative log-likelihood: -28.1611
```

4.3.3: Full posterior GP (mean + uncertainty)